

本体构建平台 Ontology API 开发指南

本文档提供了本体服务的两种使用方式：

- 1. HTTP REST API：供外部系统通过 HTTP 请求调用
- 2. 函数 (Function)：用于在平台内置函数中编辑本体数据

第一部分：HTTP REST API 接口文档

API 概述

HTTP REST API 提供了标准的 HTTP 接口，供外部系统集成使用。所有接口支持 JSON 格式的请求和响应。

基础信息：

- 基础 URL： `https://api.onto.com`
- 认证方式：Bearer Token（在请求头中传递）
- 请求格式：JSON
- 响应格式：JSON

API 认证

所有 API 请求需要在请求头中包含认证信息：

http

代码块

```
1 Authorization: Bearer YOUR_ACCESS_TOKEN
2 Content-Type: application/json
```

API 接口列表

API 1. 创建对象实例

接口： `POST /api/ontology/object-instances`

请求体：

json

代码块

```
2  "api_identifier": "本体模型api标识.对象类型的api标识",
3  "display_name": "无人机",
4  "model": "String",
5  "battery_level": 85.5
6  }
```

使用场景：接收外部传感器上报的实时观测数据并入库；用户通过前端表单手动录入新的资产设备。

响应：

json

代码块

```
1  {
2  "id": "obj_123456",
3  "api_identifier": "本体模型api标识.对象类型的api标识",
4  "display_name": "无人机",
5  "model": "String",
6  "battery_level": 85.5,
7  "created_at": "2025-01-15T10:30:00Z"
8  }
```

API 2. 删除对象实例

接口： `DELETE /api/ontology/object-instances/{id}`

路径参数：

- `id` : 对象实例 ID

请求体：

json

代码块

```
1  {
2  "api_identifier": "本体模型api标识.对象类型的api标识"
3  }
```

使用场景：设备退役销毁；手动清除录入错误的垃圾测试数据。

响应：

json

代码块

```
1  {
2    "success": true,
3    "message": "对象实例已删除"
4  }
```

API 3. 更新对象实例属性

接口: `PUT /api/ontology/object-instances/{id}`

路径参数:

- `id`: 对象实例 ID

请求体:

json

代码块

```
1  {
2    "api_identifier": "本体模型api标识.对象类型的api标识",
3    "name": "更新后的名称",
4    "status": "OK"
5  }
```

使用场景: 更新设备当前的电量、状态等动态参数; 修改业务资产的归口管理部门或负责人信息。

响应:

json

代码块

```
1  {
2    "id": "obj_123456",
3    "api_identifier": "本体模型api标识.对象类型的api标识",
4    "name": "更新后的名称",
5    "status": "OK",
6    "updated_at": "2025-01-15T10:35:00Z"
7  }
```

API 4. 查询对象实例列表

接口： `POST /api/ontology/object-instances/query`

请求体：

json

代码块

```
1  {
2    "api_identifier": "本体模型api标识.对象类型的api标识",
3    "filters": {
4      "id": "12222",
5      "name": "Double",
6      "status": "OK"
7    }
8  }
```

响应：

json

代码块

```
1  {
2    "data": [
3      {
4        "id": "obj_12222",
5        "api_identifier": "本体模型api标识.对象类型的api标识",
6        "name": "Double",
7        "status": "OK"
8      }
9    ],
10   "total": 1
11 }
```

使用场景： 管理后台展示某一类对象的完整列表；支持前端的分页展示需求。

API 5. 批量删除对象实例

接口： `POST /api/ontology/object-instances/batch-delete`

请求体：

json

代码块

```

1  [
2  {
3      "api_identifier": "本体模型api标识.对象类型的api标识",
4      "id": "12222",
5      "status": "Completed"
6  },
7  {
8      "api_identifier": "本体模型api标识.对象类型的api标识",
9      "id": "13333",
10     "status": "Completed"
11  }
12 ]

```

使用场景：对过期的历史任务数据进行一键清理；通过后台任务定时清除无效的临时对象。

响应：

json

代码块

```

1  {
2      "success": true,
3      "deleted_count": 2
4  }

```

API 6. 创建对象实例关联

接口： `POST /api/ontology/object-instances/links`

请求体：

json

代码块

```

1  {
2      "source_api_identifier": "本体模型api标识.对象类型的api标识",
3      "target_api_identifier": "本体模型api标识.对象类型的api标识",
4      "sourceObjectKey": "id_A",
5      "targetObjectKey": "id_B"
6  }

```

响应：

json

代码块

```
1  {
2    "success": true,
3    "link_id": "link_123456"
4  }
```

使用场景：建立“无人机”与“所属站点”的业务绑定关系；将“维修记录”挂载到具体的“设备”上。

API 7. 删除对象实例关联

接口： `DELETE /api/ontology/object-instances/links`

请求体：

json

代码块

```
1  {
2    "source_api_identifier": "本体模型api标识.对象类型的api标识",
3    "target_api_identifier": "本体模型api标识.对象类型的api标识",
4    "sourceObjectKey": "id_A",
5    "targetObjectKey": "id_B"
6  }
```

响应：

json

代码块

```
1  {
2    "success": true,
3    "message": "关联已删除"
4  }
```

使用场景：员工调岗解除与原部门的“所属”关系；更正由于操作失误建立的错误关联。

API 8. 复杂对象实例检索

接口： `POST /api/ontology/object-instances/search`

请求体：

json

代码块

```
1  {
2    "api_identifier": "XXXXXXModel.order",
3    "filter": {
4      "status": { "eq": "Pending" },
5      "amount": { "gt": 1000 },
6      "create_time": { "range": ["2024-01-01", "2024-12-31"] }
7    },
8    "sort": { "create_time": "desc" },
9    "limit": 50
10 }
```

使用场景：精准检索（例如：找出所有“北京地区”且“剩余电量低于20%”的“小型无人机”）。

响应：

json

代码块

```
1  {
2    "data": [...],
3    "total": 150,
4    "limit": 50,
5    "offset": 0
6  }
```

API 9. 多对象实例关联查询 (Graph Traversal)

接口： `POST /api/ontology/object-instances/traverse`

请求体：

json

代码块

```
1  {
2    "start_object_id": "AF-101",
3    "start_api_identifier": "MaritimeOntology.Unit",
```

```

4    "link_types": ["MemberOf", "FlagshipOf"],
5    "direction": "outbound",
6    "max_depth": 2,
7    "include_start": true
8  }

```

使用场景：拓扑追踪与关联追踪。例如：从某架无人机出发，查询其所属的编队，以及该编队中所有的其他关联目标。

响应：

json

代码块

```

1  {
2    "nodes_by_depth": {
3      "depth_0": [
4        { "object_id": "AF-101", "object_type": "Unit", "depth": 0 }
5      ],
6      "depth_1": [
7        { "object_id": "CSG-5", "object_type": "Group", "depth": 1 }
8      ],
9      "depth_2": [
10       { "object_id": "CVN-78", "object_type": "Unit", "depth": 2 }
11     ]
12   },
13   "total_nodes": 3,
14   "max_depth": 2
15 }

```

API 10. 聚合统计

接口： `POST /api/ontology/object-instances/aggregate`

请求体：

json

代码块

```

1  {
2    "api_identifier": "XXXXXXModel.order",
3    "group_by": ["status", "region"],
4    "metrics": {
5      "total_amount": { "sum": "amount" },

```

```
6      "order_count": "count"
7    }
8  }
```

响应：

json

代码块

```
1  {
2    "results": [
3      {
4        "status": "Pending",
5        "region": "North",
6        "total_amount": 50000,
7        "order_count": 10
8      }
9    ]
10 }
```

使用场景：汇总统计（例如：按“在线状态”汇总各型号设备的分布总数；统计各区域资产的资产累计总价值）。

API 11. 执行业务行为 (Action)

接口： `POST /api/ontology/actions/execute`

说明：Action 只能通过 HTTP API 调用，不能在函数（Function）中使用。

请求体：

json

代码块

```
1  {
2    "action_api_identifier": "dispatch_order",
3    "order_id": "ORD-2025-001",
4    "target_warehouse": "WH-CN-BJ-01",
5    "priority": "High"
6  }
```

响应：

json

代码块

```
1  {
2    "success": true,
3    "action_id": "action_123456",
4    "result": {
5      "order_status": "dispatched",
6      "estimated_delivery": "2025-01-20"
7    }
8  }
```

使用场景：触发外部系统逻辑。例如：由大模型确认指令后，调用 API 触发无人机“立即返航”的硬件控制逻辑。

API 12. 查询对象类型元数据 (Metadata Query)

接口： `GET /api/ontology/metadata/object-types/{api_name}`

说明：获取指定对象类型的完整定义（Schema），包括属性列表和关联关系。常用于为大模型（LLM）提供建模上下文。

路径参数：

- `api_name` : 对象类型的 API 标识符（如 `uav_device`）

响应：

json

代码块

```
1  {
2    "rid": "ri.ontology.main.object-type.uav_device",
3    "apiName": "uav_device",
4    "displayName": "无人机",
5    "properties": [
6      { "apiName": "id", "displayName": "ID", "type": "String", "isPrimaryKey":
7        true },
8      { "apiName": "model", "displayName": "型号", "type": "String" }
9    ],
10   "linkTypes": [
11     { "apiName": "belongs_to", "displayName": "所属单位", "cardinality": "1:N" }
12   ]
13 }
```

使用场景： 获取指定对象类型的完整定义（Schema），包括属性列表和关联关系。常用于为大模型（LLM）提供建模上下文。

API 13. 上传附件 (Upload Attachment)

接口： `POST /api/ontology/object-instances/{id}/attachments`

说明： 为特定的对象实例上传关联的附件（如图片、PDF 文档等）。

路径参数：

- `id` : 对象实例 ID

请求体 (multipart/form-data)：

- `file` : 待上传的文件流

响应：

json

代码块

```
1  {
2    "success": true,
3    "attachment_id": "att_789012",
4    "filename": "drone_photo_01.jpg",
5    "url": "https://storage.onto.com/files/att_789012"
6  }
```

使用场景： 为无人机巡检记录上传现场抓拍的图片；为关键资产设备绑定 PDF 格式的电子维修手册。

API 14. 查询场景/全局元数据（供大模型理解）

接口： `GET /api/ontology/metadata`

说明： 按 场景 或 全部场景（全局） 粒度，按需返回本体元数据，包括对象类型、链接类型、行为类型（Action）及描述。便于大模型在调用实例级 API 前，先理解「当前有哪些对象类型、它们如何通过链接关联、哪些对象类型上有哪些可执行行为」，用于规划查询与调用。

查询参数：

参数	类型	必填	说明
scene	string	是	场景：传场景 ID（如 military_expo、contract_review）表示仅该场景下的元数据；传 all 表示全部场景/全局本体。
types	string	否	请求的元数据类型，多个用逗号分隔，不传则返回全部类型。可选值： object_types 对象类型列表； link_types 链接类型（对象类型与对象类型之间的关联）； actions 行为类型（各对象类型上可用的 Action 及描述）。

请求示例：

http

代码块

```
1  # 获取某场景下的全部元数据（对象类型 + 链接类型 + 行为类型）
2  GET /api/ontology/metadata?scene=military_expo
3
4  # 仅获取全局的对象类型与链接类型
5  GET /api/ontology/metadata?scene=all&types=object_types,link_types
6
7  # 仅获取某场景下各对象类型可用的 Action 及描述
8  GET /api/ontology/metadata?scene=naval_battle&types=actions
```

响应：根据 types 返回对应字段（未请求的类型不返回或为空）。

json

代码块

```
1  {
2    "scene": "military_expo",
3    "object_types": [
4      {
5        "apiName": "Unit",
6        "displayName": "作战单元",
7        "description": "舰艇、飞机等作战单元",
8        "properties": [
9          { "apiName": "id", "displayName": "ID", "type": "String",
10            "isPrimaryKey": true },
11          { "apiName": "type", "displayName": "类型", "type": "String" }
12        ],
13        {
14          "apiName": "Group",
15          "displayName": "编队",
16          "description": "作战编队",
17          "properties": [
18            { "apiName": "id", "displayName": "ID", "type": "String",
19              "isPrimaryKey": true },
20            { "apiName": "name", "displayName": "名称", "type": "String" }
21          ]
22        },
23        "link_types": [
24          {
25            "apiName": "MemberOf",
26            "displayName": "隶属于",
27            "sourceObjectType": "Unit",
28            "targetObjectType": "Group",
29            "cardinality": "N:1",
30            "description": "作战单元隶属于某编队"
31          }
32        ],
33        "actions": [
34          {
35            "actionId": "act-return-base",
36            "displayName": "立即返航",
37            "description": "对该单元下发返航指令，适用于无人机/舰艇等",
38            "targetObjectType": "Unit",
39            "parameters": [
40              { "name": "unit_id", "type": "String", "required": true, "description":
41                "目标单元 ID" },
42              { "name": "priority", "type": "String", "required": false,
43                "description": "优先级" }
44            ]
45          },
46        ]
47      },
48    ]
49  }
```

```

44     {
45         "actionId": "act-generate-report",
46         "displayName": "生成态势报告",
47         "description": "基于当前编队生成态势报告",
48         "targetObjectType": "Group",
49         "parameters": [
50             { "name": "group_id", "type": "String", "required": true,
51               "description": "编队 ID" }
52         ]
53     }
54 }

```

说明：

- `object_types`：当前场景下的对象类型列表；每项含 `apiName`、`displayName`、可选 `description`、`properties`（可简化只列关键属性或与 API 12 一致）。
 - `link_types`：对象类型之间的链接类型；`sourceObjectType` / `targetObjectType` 表示「从哪类对象指向哪类对象」。
 - `actions`：当前场景下可执行的行为；`targetObjectType` 表示该 Action 绑定在哪类对象上（全局行为可为空）；`parameters` 与 API 11 执行时的请求体字段对应，便于大模型构造调用。
- 使用场景：大模型或 Agent 在接入本体前，先调用本接口获取「有哪些对象类型、链接关系、可执行行为」，用于生成正确的查询与 Action 调用；也可作为 RAG/提示词中的结构化上下文，减少幻觉与错误调用。

HTTP 完整端到端集成示例

场景：外部资产管理系统（EAM）在同步一台新购入的“无人机”设备时，需要同时上传维护手册，并将其关联到“巡检中心”部门。

流程 1：创建无人机实例

POST /api/ontology/object-instances

json

代码块

```

1  {
2      "api_identifier": "CityOntology.UAV",
3      "display_name": "UAV-BJ-001",
4      "type": "FixedWing",
5      "buy_date": "2025-01-20"
6  }

```

响应结果示例: `{"id": "obj_uav_101", ...}`

流程 2：上传该设备的 PDF 维修手册

`POST /api/ontology/object-instances/obj_uav_101/attachments` (Multipart Form Data)

- `file`: `maintenance_manual.pdf`

流程 3：建立与所属部门的关联

`POST /api/ontology/object-instances/links`

json

代码块

```
1  {
2    "source_api_identifier": "CityOntology.UAV",
3    "target_api_identifier": "CityOntology.Department",
4    "sourceObjectKey": "obj_uav_101",
5    "targetObjectKey": "dept_inspection_center"
6  }
```

函数开发示例

使用场景对比

使用场景	推荐方式	说明
外部系统集成	HTTP REST API	标准 HTTP 接口，易于集成
前端应用调用	HTTP REST API	通过 HTTP 请求调用
函数中编辑本体数据	平台开发方法	在函数运行时环境中使用
批量数据处理	平台开发方法	在函数中处理大量数据

总结